1. *Write an SQL statement that returns the 7-day rolling incremental CAC for a given Meta campaign, excluding conversions already counted in a prior 30-day window. Explain each clause in one sentence*:

Context:

- 7-day rolling incremental CAC

Assumptions:

- One conversion row per event
- Two source tables exist (or would be created prior to the query):
    - campaign_spend: one row per day per campaign, with total spend for that day
    - conversions: one row per conversion event, with user ID, campaign, and date

```sql
-- table to filter out multiple conversions from the same user within 30 days
WITH incremental AS (
    SELECT
        c.date,
        c.campaign_id,
        c.user_id,
        c.conversion_id
    FROM conversions c
    WHERE c.campaign_id = 'META_001'
      AND NOT EXISTS (
        -- check if the same user converted in the same campaign within the last 30 days
        SELECT 1
        FROM conversions prior
        WHERE prior.user_id = c.user_id
          AND prior.campaign_id = c.campaign_id
          AND prior.date >= DATE_SUB(c.date, INTERVAL 30 DAY)
          AND prior.date < c.date
      )
),

-- table with total spend (campaign_spend table) and total incremental conversions (incremental table) per day
daily AS (
    SELECT
        s.date,
        s.spend,
        COUNT(ic.conversion_id) AS incremental_conversions
    FROM campaign_spend s
    -- left join to ensure days with zero incremental conversions are kept
    LEFT JOIN incremental ic
        ON s.date = ic.date
        AND s.campaign_id = ic.campaign_id
    WHERE s.campaign_id = 'META_001'
    GROUP BY s.date, s.spend
),

-- table with 7 days rolling sum from daily table
rolling_7d AS (
    SELECT
        date,
        SUM(spend) OVER (
            ORDER BY date
            -- 7 last days at each row (assumes one row per day)
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ) AS rolling_7d_spend,
        SUM(incremental_conversions) OVER (
            ORDER BY date
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ) AS rolling_7d_conversions
    FROM daily
)

-- CAC calculation from rolling_7d table: total spend divided by total incremental conversions per day
SELECT
    date,
    rolling_7d_spend,
    rolling_7d_conversions,
    SAFE_DIVIDE(rolling_7d_spend, rolling_7d_conversions) AS incremental_cac_7d
FROM rolling_7d
ORDER BY date DESC
```

2/3. Describe, step-by-step, how you would design a privacy-first, server-side tracking and data pipeline for a rehabilitation treatment provider that cannot rely on cookies and must protect sensitive health-adjacent user data.
Include: (a) what tools and architecture you would select, (b) how you would ensure HIPAA / GDPR / privacy-compliant consent handling, (c) how you would validate data integrity and event accuracy before data enters the warehouse, and (d) what monitoring and alerting you would implement if data loss, drift, or anomalies occur.

The starting point here is not technical. Before choosing any tool or architecture, the key question is: what data do we actually need, and what should never be collected in the first place. I have seen firsthand how sensitive health data can be. My wife is a physician and researcher who works with this type of data regularly, and that shapes how I think about the problem. With that in mind, I would approach the design in four steps.

The first is moving away from browser-side tracking entirely. The problem with traditional cookie-based tracking is that data goes straight from the user's browser to third-party tools, with no control over what leaves our site. A server-side layer fixes that: all events pass through a first-party server first, which acts as a filter before anything reaches the pipeline. For a BigQuery-first stack, the natural fit is Google Tag Manager Server-Side on Google Cloud Run. The actual implementation sits closer to engineering than analytics, so this would be done in close partnership with that team.

The second is consent. No data gets collected before the user explicitly agrees, and for a health provider that consent needs to be granular. Someone might be comfortable with basic analytics but not with marketing tracking. The consent log also needs to be fully auditable to satisfy HIPAA, GDPR, and LGPD requirements.

The third is validation before data enters BigQuery. Basic checks at ingestion: are all required fields present, is there anything that needs to be anonymised, and does the event look like real user behaviour and not a bot.

The fourth is monitoring. At other jobs I managed daily automated pipelines in BigQuery where alerts fired automatically if an update failed or gaps appeared in the tables. Same logic here: automated alerts for volume drops, unexpected shifts in event distribution, and anomalies. The goal is always for the team to be the first to know, not a stakeholder flagging something wrong in a dashboard.

4. Explain how you would eliminate a weekly manual spreadsheet process that currently requires joining paid media performance data (e.g., Meta/Facebook Ads) with downstream business outcomes (e.g., leads, conversions, or revenue) and replace it with an automated, repeatable pipeline within 10 business days. Describe the tooling you would choose, the transformation logic you'd implement, and how you would document the workflow so the broader team can rely on it.

10 business days is a realistic timeline, assuming no major blockers on the infrastructure side and here is how I would approach it.

First, understanding the current process in detail is the starting point. So, I would map the current manual process: what sources are being joined, what variables matter, and what the output looks like.

Second, bring both data sources into BigQuery. Meta Ads has a native API that can feed data directly into BigQuery. CRM data (leads, conversions, revenue) would be connected from existing source systems.

Third, apply the necessary ETL logic: remove duplicates, standardise fields and data types, join both sources by common variables (date and campaign ID, for example), and calculate derived metrics like ROAS or CAC.

Fourth, build a clean output table in BigQuery as the single source of truth, then connect Looker on top of it, so the team can both use the dashboard and run ad hoc queries directly on the table without needing to export anything. I would also set up automated alerts following the same logic from question 3, to catch pipeline failures before they affect the output.

On documentation: I'd keep a shared Google Doc covering data sources (connections, endpoints, refresh schedule), ETL logic and methodology, the BigQuery table structure for anyone needing to run ad hoc analysis, alert instructions in case something breaks, known limitations and edge cases, pipeline ownership, and a simple flow diagram showing how data moves from source to database to dashboard.

5. A senior client insists on moving to last-click attribution. Your analysis shows the actual multi-touch model inflates ROAS by 28%. How would you (a) structure the conversation, (b) present the new model, and (c) implement it without delaying their next budget cycle?

I never present a problem, especially to a senior stakeholder, without a solution. So before the conversation, I would prepare a full analysis with the corrected numbers ready to present. The goal is to frame it as "I found something that may be affecting the numbers and I want to show you before your next budget cycle."

I would build a historical rebase with three scenarios side by side: ROAS as currently calculated, ROAS recalculated with the corrected multi-touch model, and ROAS recalculated with last-click. Having the three numbers side by side makes the impact immediately visible without requiring the client to take anything on faith. If historical data is not available in the database, I would schedule the presentation as far in advance as possible to allow enough data to accumulate to build the comparison, while still leaving time before the client's budget cycle.

With the three models already built in point (b), the comparison is ready for the client's decision. I would suggest implementing one of the two correct models right away, keeping the current one in parallel if needed for internal alignment on the client's side, until their internal approval meeting. A clear deadline to shut it down would be agreed upfront to avoid duplicating work indefinitely.

6. What model would you build to forecast a client's next-month spend potential based on past performance, seasonality, and external signals? Outline features, target variable, and evaluation metric.

I would build a linear regression model to predict next-month spend potential (the spend level that would maximise ROAS given the client's historical performance).

For input variables, I would look at average monthly spend over the last 3, 6 and 12 months (using averages rather than totals to keep input variables in the same unit as the target), historical ROAS by period, and month-over-month growth trend, as past performance. For seasonality, I would calculate an index measuring how each month deviates from the overall baseline, which requires at least 2 years of history to have more than one observation per month. If that is not available, an alternative is to work at a weekly level, comparing the same week position within each month (week 1, 2, 3) across different months, which increases the number of observations. Either way, before calculating the index I would treat outliers from non-recurring events (promotions, stock clearance, etc.) using historical records, visual inspection of the series, and standard deviation thresholds. Google Trends for relevant sector terms, consumer confidence index, and key calendar events for the client's industry would cover external signals.

For evaluation, I would use MAPE (Mean Absolute Percentage Error). To calculate it, I would train the model on data up to a certain point, project the following months that already have known results, and compare projected vs actual, keeping those months out of the training set to avoid data leakage.

7. How do you build a proactive analytics organization instead of a reactive ticket machine? Be specific about systems, ownership lines, alerting/monitoring, and how you ensure teams aren't dependent on others to flag issues.

For me the difference between a proactive and a reactive analytics team is not tooling but ownership, visibility, and discipline.

Every metric, pipeline, and dashboard needs a named owner within the team, someone responsible for monitoring it day to day. But operational ownership alone does not scale because the volume of databases, flows and dashboards in a growing organisation is often too high for any individual to monitor manually, and that is when things start to fall through the cracks.

To make ownership scalable, I would set up a centralised control tower: a monitoring dashboard showing the real-time status of all pipelines, flows and dashboards in one place. Combined with automated alerts for volume drops, data gaps or anomalies, the team can cover a large surface area without relying on manual checks or waiting for a stakeholder to notice something is off. The goal is for the team to always be the first to know.

On the team side, I believe in short and frequent touchpoints rather than long alignment meetings. A daily sync of no more than 30 minutes, where each person has a couple of minutes to share status and flag blockers, keeps everyone aligned without consuming the day. Tools like Jira or Monday help analysts stay clear on their priorities and give leads full visibility on what is in progress, without needing to ask.

Autonomy is also part of what makes a team proactive. If analysts need approval for every decision, they become a ticket machine themselves. When people know the boundaries of their autonomy, they stop waiting for permission and start moving. The boundary is impact. Low stakes decisions the analyst owns. As the potential reach widens, other teams, the business, the client, the level of sign-off required goes up. Done right, it accelerates decisions rather than slowing them down.

The same logic applies in the other direction. Reducing the number of basic questions that reach the analytics team is also critical, as it usually takes a disproportionate amount of the team's time and

energy. Users will always find it faster to ask than to read documentation, so the documentation needs to come to them. Inline instructions in dashboards, tooltips on key metrics, links to short reference docs rather than extensive manuals, visual flow diagrams where relevant, and a FAQ covering the most common questions all help reduce that dependency. That is time the team can spend finding the next problem before it finds them.

8. What standards, governance, and rituals do you put in place so analytics work is consistently high-quality, not dependent on heroics?

For analytics work to be consistently good, "good" needs to mean the same thing to everyone on the team. That starts with having clear, shared definitions: every metric has one definition, documented and accessible, so the same number does not mean two different things to me and you. Data transformation rules are explicit and applied consistently, raw data is never consumed directly, and every pipeline or analysis that goes to production comes with documentation. Without that, quality becomes dependent on whoever built it, and that is exactly the heroics problem.

On governance, one thing I always insist on is that ownership needs to be attached to roles, not individuals. Accountability should not walk out the door when someone leaves.

Beyond ownership, quality without measurement is just an opinion, and we all have our own opinions. I find it more useful to think about it in concrete terms: accuracy, completeness, and on-time execution, think of it as an analytical OTIF. That gives the team a proactive view of how the work is holding up, not just whether we crossed the finish line.

Decision authority also needs to be proportional to impact. Every delivery follows a clear workflow, with the right people signing off at the right stages. I learned this the hard way on a project where governance existed on paper but was not structured at the right milestones, and when something went wrong, there was no trail to understand where the process had failed. Good governance is what creates that trail.

On rituals, the thing they actually protect against is quality drift. A periodic quality review of what is live ensures that what the team built is still correct, still being used, and still solving the problem it was designed for. I strongly believe that better no data than wrong data, and with many projects running in parallel it is easy for something to silently break while teams downstream are still depending on it.

Post mortems after significant deliveries, and also after failures, help the team learn from what went right and what did not, without blame. The goal is to fix the process, not the person.

And periodically, it is worth questioning whether a solution built six or twelve months ago is still the best answer to the problem, or whether something better has emerged since. If the bar moves, the solution should move with it.

Finally, a structured review with key stakeholders every quarter brings an outside perspective on whether the work is still solving the right problems. Day-to-day issues have their own channels, this is about the bigger picture.

9. You need to build an analytics team that actually works. Who do you hire first, how do you sequence headcount, what do you outsource vs own, and how do you structure accountability so analytics owns outcomes — not just dashboards?

The first hire depends on the biggest gap, and in a greenfield scenario that gap is almost always infrastructure. Before anyone can analyse data or build dashboards, the data needs to exist, flow reliably, and be structured in a way that others can work with. So the first role I would hire is someone who can build that foundation: data architecture, external connections, warehouse design, and the best practices that everything else will be built on top of.

Once the foundation is in place, the next hire focuses on working the data: ETL, table and database creation, and building the first dashboards. Only after that does it make sense to bring in someone more analytically focused, someone who can surface insights, interpret results, and translate data into decisions. Hiring in the wrong order means analysts with nothing reliable to analyse, or dashboards built on top of data that was never properly structured.

On outsource vs own, I think about it practically. If I have five roles to cover and budget for three, something has to give. Either I find someone who can genuinely cover two roles without losing quality, or I outsource what I cannot staff internally. That tradeoff needs to be conscious, not accidental. On scope, my instinct is that the analytics team should own everything related to understanding and using data: architecture, models, insights. What I would look to the engineering team for is the infrastructure layer, APIs, server-side integrations, the plumbing that moves data from one place to another. The line is not always clean, but that is the principle I work from.

The hardest part is making sure the team is invested in what happens after the delivery. The way I think about this is simple: if the business team has a metric in their annual goals, and my team has a project that directly impacts that metric, that metric goes into my team's goals too. At a previous job, we built an optimisation tool where one of the key outcomes was operational productivity. We put that same productivity metric in our team's annual targets. That changed everything. The team did not just build and hand over the tool, they kept an eye on adoption, monitored the analytical outputs, and flagged when the numbers suggested something was off. The operational decisions still belonged to the business team. But analytics stayed close enough to know when the tool needed to evolve. That is the difference between a team that ships and forgets and one that stays accountable for what it builds.